Restaurant Menu API — Assessment Set A6

Web Development Assessment: Backend API without a Database

Total: 100 marks (100 automated test cases, 1 mark each)

1. Problem statement

A restaurant group needs a back-end service to manage its menu items. Build a REST API for a **Restaurant Menu** resource from scratch. You may use **Express (Node.js)** or **Spring Boot (Maven)**. There is no starter code.

Data is kept **in memory only** (an array or list inside your program). Do not use a database, a JSON file or any seed data: when your server starts, it must be **empty**. The evaluator will create every record it needs by calling your `POST` endpoint.

2. Ground rules

1. Build from scratch. Only the API is required, no frontend.
2. Your server must read its port from the `PORT` environment variable (default 8080 if it is not set).
3. Every request and response body is JSON.
4. Restart your server before each evaluation run so the data starts empty.
5. Do not change field names, paths or status codes. They are checked exactly.

3. Resource: `dishes`

Field	Type	Rules	Required
`id`	server-generated	Read-only. Created by your server, unique, number or string. Never taken from the request body.	—
`dishName`	string	Trimmed; 3 to 60 characters. Searchable and sortable.	Yes
`restaurant`	string	Trimmed; 2 to 40 characters. Searchable.	Yes
`category`	string (enum)	Exactly one of: `Starter`, `MainCourse`, `Dessert`, `Beverage`, `Snack`. Case-sensitive. Filterable.	Yes
`price`	number	Greater than 0 and at most 5000. Decimals allowed. Range-filterable and sortable.	Yes
`calories`	integer	Whole number from 0 to 5000. Sortable.	Yes
`isVegetarian`	boolean	`true` or `false` (a JSON boolean, not a string). Filterable.	Yes

Unknown fields sent in a request body are ignored (they are not stored and not returned).

Sample record you would send with `POST`:

```
{"dishName": "Paneer Tikka",
  "restaurant": "Spice Garden",
  "category": "Starter", "price": 220,
  "calories": 320, "isVegetarian": true}
```

Sample record your server returns:

```
{"id": 1, "dishName": "Paneer Tikka",
  "restaurant": "Spice Garden",
  "category": "Starter", "price": 220,
  "calories": 320, "isVegetarian": true}
```

4. Endpoints

Method	Path	Purpose	Success
POST	`/api/dishes`	Create a record	`201` + the created record
GET	`/api/dishes`	List records (search, filter, sort, pagination)	`200` + list response
GET	`/api/dishes/{id}`	Get one record	`200` + the record
PUT	`/api/dishes/{id}`	Replace a record (all required fields)	`200` + the updated record
PATCH	`/api/dishes/{id}`	Update some fields	`200` + the updated record
DELETE	`/api/dishes/{id}`	Delete a record	`204` with an empty body

- `PUT` replaces the whole record and requires every required field. It never creates a record: an unknown id returns `404`.
- `PATCH` needs at least one recognised field (an empty body `{}` is `400`). Every field you send is validated by the same rules as in `POST`.
- `id` can never be changed, even if the request body contains one.

5. List query parameters (`GET /api/dishes`)

Parameter	Meaning	Rules
`page`	Page number	Positive integer. Default 1. Anything else is `400`. A page past the last one returns `200` with empty `data`.
`limit`	Items per page	Positive integer, **maximum 50**. Default `10`. Anything else is `400`.
`search`	Text search	Case-insensitive "contains" match on `dishName` **or** `restaurant`. Empty value means no search.
`category`	Filter by enum	Exact value from the list above. An unknown value is `400`.
`isVegetarian`	Filter by boolean	`true` or `false`. Anything else is `400`.
`minPrice`	Lowest `price`	Inclusive. Must be a number, otherwise `400`.
`maxPrice`	Highest `price`	Inclusive. Must be a number, otherwise `400`. If `minPrice` is greater than `maxPrice`, return an empty result with `200`.
`sort`	Sort field	One of `dishName`, `price`, `calories`. Anything else is `400`. Text is compared ignoring letter case.
`order`	Sort direction	`asc` (default) or `desc`. Anything else is `400`.

How a list request is processed: **filters -> search -> sort -> pagination**. All filters and the search are combined with AND. Without `sort`, records come back in the order they were created.

List response:

```
{ "data": [
  {"id": 1, "dishName": "Paneer Tikka",
    "restaurant": "Spice Garden",
    "category": "Starter", "price": 220,
    "calories": 320, "isVegetarian": true}
  ],
  "page": 1, "limit": 10, "total": 1, "totalPages": 1 }
```

`totalPages` is `ceil(total / limit)`, and `0` when `total` is `0`. `total` is the number of records **after** filters and search, before pagination.

> **Short on time? Minimum list.** If you do not finish pagination, filtering, search and sort, still return `{ "data": [the first 10 records in creation order], "page": 1, "limit": 10, "total": <number of records>, "totalPages": <ceil(total / 10)> }` from `GET /api/dishes`. This alone earns the **List basics** marks (8 marks), and the other **60 marks** (every group except Pagination, Filter, Search, Sort and Combined queries) do not depend on these features.

6. Errors and status codes

Every error response must be JSON with a short text message under the key `error` **or** `message`, for example `{"error": "Record not found"}` or `{"message": "Record not found"}`. The **wording is free** and is not checked. The **status code is checked exactly**.

Situation	Status
Created	201
Read, list, update	200
Deleted	204 (empty body)
Invalid body, invalid field, malformed JSON, invalid query parameter	400
Unknown id, or unknown route	404
Duplicate unique value	409

Validation: strings must be JSON strings (trimmed; blank after trimming is invalid), numbers must be JSON numbers, booleans must be JSON booleans (`"true"` as text is invalid), integers must be whole numbers, and enums must match exactly.

- This resource has no unique field. Two records with identical values are allowed. Your server must never crash or return an HTML error page, even for malformed JSON.

Rules that are easy to miss

1. An id that does not exist returns `404`, whatever it looks like (`/api/dishes/987654`, `/api/dishes/abc`). Never `400` and never `500`.
2. Text is trimmed before it is checked and before it is stored: `"  Paneer Tikka  "` is saved as `"Paneer Tikka"`.
3. A number sent as text (`"220"`) is invalid. So is `isVegetarian` sent as `"true"`, and `calories` sent as `1.5`.
4. `price` of `0` or a negative number is invalid.
5. `POST` ignores any `id` you send. `PUT` and `PATCH` never change the id.
6. `PUT` on an unknown id is `404`. It does **not** create a record.
7. `PATCH` with `{}` is `400`. `PATCH` leaves every field you did not send unchanged.
8. `DELETE` returns `204` with **no body**. Deleting the same id again returns `404`.
9. A page past the last page is **not** an error: return `200` with `"data": []`.
10. `minPrice` greater than `maxPrice` is **not** an error: return `200` with `"data": []`, `total 0`, `totalPages 0`.
11. Without `sort`, records come back in the order they were created (oldest first).
12. Query values you do not support (`sort=banana`, `order=sideways`, `isVegetarian=maybe`, a category that is not in the list) are `400`.

7. Examples

All examples use the base path `/api/dishes`. Values are illustrative.

Create, invalid (`price` is 0): `POST /api/dishes` returns `400`

```
{ "error": "price must be greater than 0" }
```

Get one, unknown id: `GET /api/dishes/987654` returns `404`

```
{ "error": "Record not found" }
```

Filter, sort and paginate: `GET /api/dishes?category=Starter&isVegetarian=true&sort=price&order=desc&page=1&limit=2` returns the 2 records with the highest `price` among those whose `category` is `Starter` and `isVegetarian` is `true`. `total` is the number of matching records, not 2.

```
{ "data": [
    {"id": 3, "dishName": "Paneer Tikka",
      "restaurant": "Spice Garden",
      "category": "Starter", "price": 220,
      "calories": 320, "isVegetarian": true}
  ],
  "page": 1, "limit": 2, "total": 5, "totalPages": 3 }
```

Only one record is shown to keep the example short: `data` holds up to `limit` records.

Search: `GET /api/dishes?search=paneer` returns every record whose `dishName` or `restaurant` contains that text in any letter case.

Range: `GET /api/dishes?minPrice=110&maxPrice=440` returns records with `price` from 110 to 440, both ends included.

8. How your work is evaluated

- Your server is started **empty**. The evaluator then creates its own records by calling `POST /api/dishes` (about 45 records with random values, **not** the sample above), and runs **100 automated checks** on every endpoint in a fixed order.
- Each check is worth **1 mark**. There are no part marks inside a check.
- Checks look at the HTTP status code, the JSON shape and the field values. They do not look at your code, folder structure, language or framework (Express and Spring Boot are marked the same way).
- Error wording is not checked (see section 6). Status codes and success bodies are checked exactly.
- If `POST` does not work, most other checks cannot run and their marks are lost. **Build and test `POST` first.**
- If your server crashes while the checks run, every check after the crash fails. Handle bad input without crashing.
- Each request must answer within 5 seconds. A full run is limited to 3 minutes.
- Because data is random, you cannot pass by hard-coding answers.

Suggested build order: `POST` and `GET /{id}`, then `GET` list, `PUT`, `PATCH`, `DELETE`, then pagination, filters, search and sort, then a pass through the "rules that are easy to miss".

9. Run and submit (assessment day)

1. Start your server so that it listens on the port in `PORT` (default 8080) with **no data** in it.
2. In the assessment kit folder, run `npm run whoami` once to confirm your set.
3. When your API is ready, run `npm run evaluate`. If your server is not on `http://localhost:8080`, add `-- --url http://localhost:PORT`.
4. The evaluator shows each test case as PASS or FAIL but does **not** show marks. When it finishes you will see a message that your submission was recorded. Your result goes to your trainer.
5. If you see "not recorded yet", keep the window open, tell the invigilator and run `npm run submit-pending` when asked.
6. Restart your server before every run. Every run is recorded.
7. If the evaluator stops with a message that your server cannot be reached or is not empty, nothing was submitted: fix that and run again.

10. Marking scheme

Group	What is checked	Marks
Create (POST)	Valid create returns 201 with an id and the stored values (text trimmed); rejects missing fields, wrong types, out-of-range numbers, short text, bad enum values, empty body and malformed JSON with 400; duplicate rule for the unique field.	14
Read one (GET /:id)	Returns the right record for each id; 404 with a JSON error body for unknown ids, including ids that do not look like ids.	8
List basics (GET)	200 and the list shape (data, page, limit, total, totalPages); defaults page 1 and limit 10; correct total; every item well formed; creation order by default.	8
Update (PUT)	Full replace returns 200 with new values and the same id; 400 for missing or invalid fields; 404 for unknown id (nothing is created); other records untouched; id in the body ignored; duplicate rule.	10
Update (PATCH)	Partial update changes only the fields sent; 400 for an empty body, invalid values and wrong types; 404 for unknown id; unknown fields ignored; id cannot change; duplicate rule.	10
Delete (DELETE)	204 with an empty body; the record is gone afterwards; second delete and unknown id return 404; total drops by exactly one; other records unaffected.	8
Pagination	limit and page give the right slices without overlap; last page holds the remainder; totalPages; every record appears exactly once across pages; page past the end is empty; invalid page and limit values return 400; limit above 50 returns 400.	12
Filter	Enum filter, true/false filter and the min/max range (inclusive) return exactly the matching records; filters combine with AND and with pagination; invalid filter values return 400.	10
Search	Case-insensitive partial match on the two text fields; no match gives an empty list with total 0; works with pagination; empty search means no search.	8
Sort	Ascending and descending order on the allowed fields (numbers sorted as numbers, text ignoring case); invalid sort field or order returns 400.	6
Combined queries	Search, filter, sort and pagination together give the exact expected page; min greater than max gives an empty result with 200.	4
HTTP basics	JSON content type on every response and a JSON 404 for unknown routes.	2
Total		**100**

11. Project setup

Set up and run your API project first. Then go to section 12 to run the evaluator.

> **Node.js is mandatory.** The evaluator runs on Node.js, so it must be installed on your laptop even if you build your API in Spring Boot.

1. **Windows only:** allow npm commands to run in the VS Code terminal (PowerShell). Run this once:
   ```
   Set-ExecutionPolicy RemoteSigned -Scope CurrentUser
   ```
   If it asks for confirmation, type `Y` and press Enter.
2. Check that Node.js is installed:
   ```
   node -v
   ```
   It should show version **18 or newer**. If it does not, install Node.js first.
3. **Spring Boot students only:** check that Java and Maven are installed:
   ```
   java -version
   javac -version
   mvn -v
   ```
   If `mvn -v` is not recognised, install Maven, then close and reopen the terminal:
   ```
   winget install Apache.Maven
   ```
4. Create your project:

Express students: creating your project

1. Create a new folder for your API, **not** inside the kit. Open it in a second VS Code window and open a terminal there.
2. Create the project and install Express:
   ```
   npm init -y
   npm i express
   ```
3. Create `index.js` and make your server listen on `process.env.PORT || 8080`.
4. Run your API from your project's terminal:
   ```
   node index.js
   ```

Spring Boot students: creating your project

1. Go to https://start.spring.io and choose: Project **Maven**, Language **Java**, the default Spring Boot version, Packaging **Jar**, and the Java version that `java -version` showed (for example **17** or **21**).
2. Click **Add Dependencies** and add **Spring Web**. Do not add any database dependency.
3. Click **Generate**. Unzip the downloaded file into its own folder, **not** inside the kit, and open that folder in a second VS Code window.
4. In `src/main/resources/application.properties`, add:
   ```
   server.port=${PORT:8080}
   ```
5. Run your API from your project's terminal:
   ```
   mvn spring-boot:run
   ```

12. Running the evaluator (auto-evaluation)

Follow the steps below to run the evaluator for your assessment.

1. Unzip the assessment kit (`assessment-kit.zip`) into a new folder. Download it from: https://drive.google.com/drive/folders/1vGy1F18GoaPxMhxZkW7-nzuhSlg81HsP?usp=sharing
2. Open that folder in VS Code (**File > Open Folder**), then open a terminal (**Terminal > New Terminal**). Keep your own API project in a separate folder, **not** inside the kit.
3. Log in:
   ```
   npm run whoami
   ```
   Enter your registration number, your **6-digit PIN** (printed on your slip) and the set printed on this question paper: **A6**.
4. Start your API on port **8080** with **no data** in it (restart it just before the run), then run the evaluator in the kit's terminal:
   ```
   npm run evaluate
   ```

The evaluator lists each test case with its status, **PASS** or **FAIL**. It does not show marks. When it finishes, you will see the message **"Submission recorded"**. Wait for this message and do not stop your server while it runs.

> **Important**
> - Stay connected to the classroom WiFi until you finish.
> - Your PIN is personal and works on **one laptop only**. Do not share it.
> - If you see any other message, check the **"What the messages mean"** table in the instruction document or tell your invigilator.

All the best.